

U.S. Provisional Patent Application — Technical Specification

- Notice
- Cross-Reference to Related Applications
- Title of the Invention
- Technical Field of the Invention
- Background of the Invention
- Brief Summary of the Invention
- Brief Description of the Drawings
- Detailed Description of the Invention
- Claims
- Abstract of the Disclosure
- Reduction to Practice — Empirical Evidence
- Filing Checklist (Assignee Action Items)

U.S. Provisional Patent Application — Technical Specification

Field	Value
Title	System and Method for Geometric Repository Partitioning, Scoped Agent Bound Enforcements, and Asynchronous Cross-Boundary Transaction Synchronization in Parallel Autonomous Software Engineering Environments
Document class	Provisional patent application — technical specification (draft)
Version	2.0
Date	2026-06-18
Assignee (intended)	Neural Draft LLC
Inventor(s)	Amir Khodabakhsh (California, United States)
Status	Pro se provisional — not filed

Notice

This document is a **draft technical specification** intended to support a U.S. Provisional Patent Application under 35 U.S.C. § 111(b), filed pro se or through counsel. It is **not legal advice**. Corrected drawings (GIV terminology) are in

Gaijinn/docs/campaign/legal/provisional-filing/figures/. Supersedes patentvisuals.jpeg where that sheet incorrectly labeled **Grit** instead of **GIV (Agent Intent Vector)**.

Implementation references: aoc-cli/aoc_cli/gravity.py, aoc-cli/aoc_cli/blueprint.py, aoc-cli/aoc_cli/giv.py, aoc-cli/aoc_cli/helpers/handoff.py, aoc-cli/aoc_cli/helpers/council.py, aoc-cli/aoc_cli/helpers/merge.py, aoc-supervisor/aoc-supervisor/preflight.py.

Cross-Reference to Related Applications

[Optional — complete upon filing]

This application claims priority to U.S. Provisional Patent Application No. 62/XXX,XXX, filed [DATE], entitled “[TITLE],” the entire contents of which are incorporated herein by reference.

Title of the Invention

System and Method for Geometric Repository Partitioning, Scoped Agent Bound Enforcements, and Asynchronous Cross-Boundary Transaction Synchronization in Parallel Autonomous Software Engineering Environments

Technical Field of the Invention

The present invention relates generally to automated software development and multi-agent coordination frameworks. More specifically, the present invention relates to computer-implemented architectures for non-invasively compiling multi-module software repositories into machine-readable operational contracts, calculating discrete network-topology information bottlenecks via differential-geometry graph curvature, and enforcing strict runtime permission scopes and cross-boundary asynchronous data-flow mutations during concurrent autonomous agent execution.

Background of the Invention

State of the Art

Conventional multi-agent coding frameworks partition tasks using shallow natural-language prompts or rigid directory-tree heuristics. Version control systems (e.g., Git) track file-level history but do not model implicit functional or logical coupling across disparate modules. Static import-graph analyzers expose explicit dependencies but miss latent information bottlenecks where independent files share operational side effects, state pipelines, or deployment surfaces.

The Paradigm Flaw

When multiple autonomous coding agents execute concurrently over a shared repository, agents frequently satisfy localized acceptance criteria by modifying resources, state variables, or API endpoints controlled by sibling execution lanes. This phenomenon is referred to herein as **sibling path trespass**. Post-hoc text observation, prompt engineering, or model-dependent guardrails are non-deterministic and fail to prevent structural execution breaches before they mutate a filesystem or API surface.

The Parallel Serialization Trap

To mitigate trespass, prior art frameworks apply union-find clustering over import graphs, fusing coupled files into **atomic weld blocks**. On a representative 171-node monorepo baseline, legacy union-find welding produced **2 atomic weld blocks** with a **40-file largest cluster**, serializing **44 files** into single-threaded execution lanes. This brute-force approach destroys parallel software development return on investment by forcing agents that appear independent to run sequentially.

The Missing Metric

No prior system combines: (i) Ollivier-Ricci curvature computed via Wasserstein optimal transport over neighborhood probability distributions; (ii) per-agent immutable write-scope permission vectors enforced at merge time; and (iii) an append-only asynchronous handoff transaction bus whose synchronization invariant is a hard prerequisite to repository integration.

Brief Summary of the Invention

The disclosed system executes a closed deployment loop:

Intake Scan → Geometric Analysis → Blueprint Compilation → Isolated Worker Spawn → Collect → Validate → Merge → Governance Score → Upstream Preflight Gate

An intake scanner module transforms a target codebase into a directed interaction graph $G = (V, E)$ without modifying host-system source code. A geometric curvature engine computes structural gravity at vertices and Ollivier-Ricci curvature κ at edges using earth-mover distance between neighborhood distributions. Edges with κ below a predetermined hard floor (e.g., -0.30) are classified as shadow bridges or dark bridges.

A blueprint planner partitions the repository into isolated work units. In legacy mode, a union-find clustering pass fuses dark-bridge-connected files into atomic single-worker blocks (the **Surgery Rule**). In gateway mode, dark bridges become HANDOFF_ONLY transactional boundaries rather than atomic welds (the **Contraction Rule**), unlocking parallel lanes while routing cross-boundary mutations through a council transaction bus.

Each parallel agent workspace receives an immutable **Agent Intent Vector (GIV)** permission envelope specifying allowed write paths, sibling-denied paths, denied commands, and structural enforcement tokens. A merge integrity harness validates file deltas against the GIV, blocks trespass, verifies transaction bus synchronization, and emits an honest structural convergence score that does not pad metrics when workers produce zero fresh deltas.

Empirical reduction to practice (171-node monorepo, 2026-06-16): 4 concurrent workers, 0 trespass violations, 1.0 validation pass rate, 92 handoff gateway edges, 44 files unlocked from atomic welds, transaction bus synchronized (TX-HT-6D0B24), honest convergence 0.8889.

Brief Description of the Drawings

FIG. 1 is a structural block diagram of the deployment loop (reference numerals 100–124), illustrating the interaction between an intake scanner module (102), a geometric curvature partitioning engine (106), a blueprint planner (108), isolated agent sandboxes (114, 116), output log parsing (118), and a merge integrity harness (122).

FIG. 2 is an engineering flowchart illustrating the step-by-step mathematical transformation of directed invocation edges into discrete neighborhood probability distributions, cost matrix construction, Wasserstein optimal transport distance computation, and curvature valuation κ (reference numerals 200–213).

FIG. 3 is a structural logic diagram demonstrating the transformation of a detected structural bottleneck (dark bridge) into a `HANDOFF_ONLY` gateway edge to unlock parallel code replication, contrasting legacy union-find atomic weld blocks with gateway-mode handoff boundaries (reference numerals 300–312).

FIG. 4 is a runtime schema of the Agent Intent Vector (GIV) permission envelope, illustrating the GIV fields — worker identifier, allowed paths, denied paths, structural enforcement tokens, and invariants — together with the merge-time trespass classifier (reference numerals 400–416).

FIG. 5 is a flowchart of council transaction bus ticket emission, scaffold hardening, receipt, and synchronization (numerals 500–510).

FIG. 6 is a merge pipeline diagram showing worker deltas, GIV trespass classification, validate-worker gates, topological merge order, already-merged disposition, and honest convergence scoring (numerals 600–610).

FIG. 7 is a supervisory daemon block diagram with metrics manifest polling, authorization gate matcher, drift detection, and circuit breaker transitioning grid sprint readiness (numerals 700–712).

FIG. 8 is a non-invasive capability extraction flow from target repository through modality selection, exclude-pattern filtering, capability model writer, and normalized graph state (numerals 800–810).

FIG. 9 is a capability topology governance diagram with typed capability nodes, directed edge reachability, blueprint validator, overlap detector, and build manifest rejection (numerals 900–908).

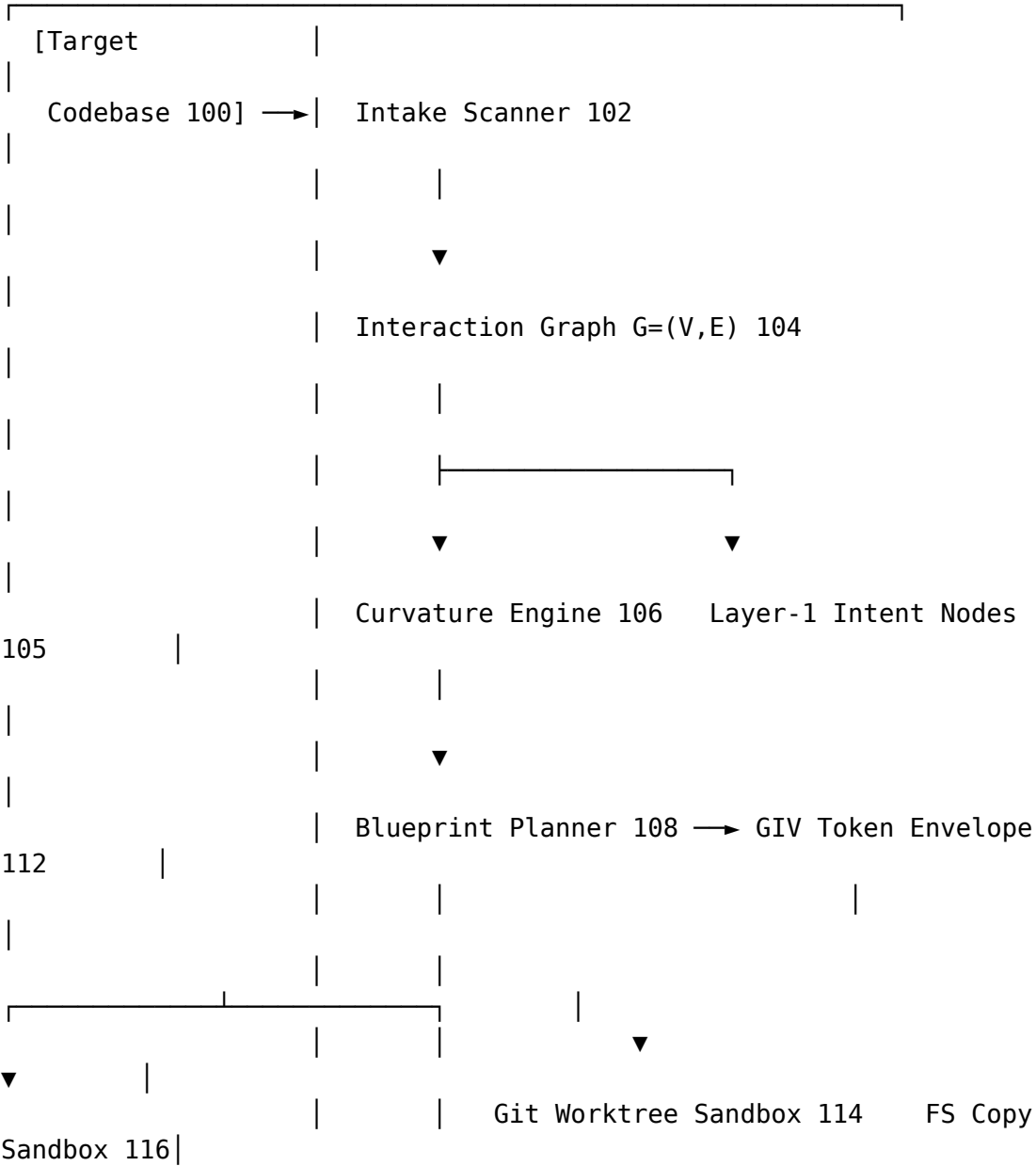
FIG. 10 is a structural gravity certification flowchart with per-node gravity score, hard-floor comparator, governance metadata flagging, human calibration review, and preflight GIV block (numerals 1000–1008).

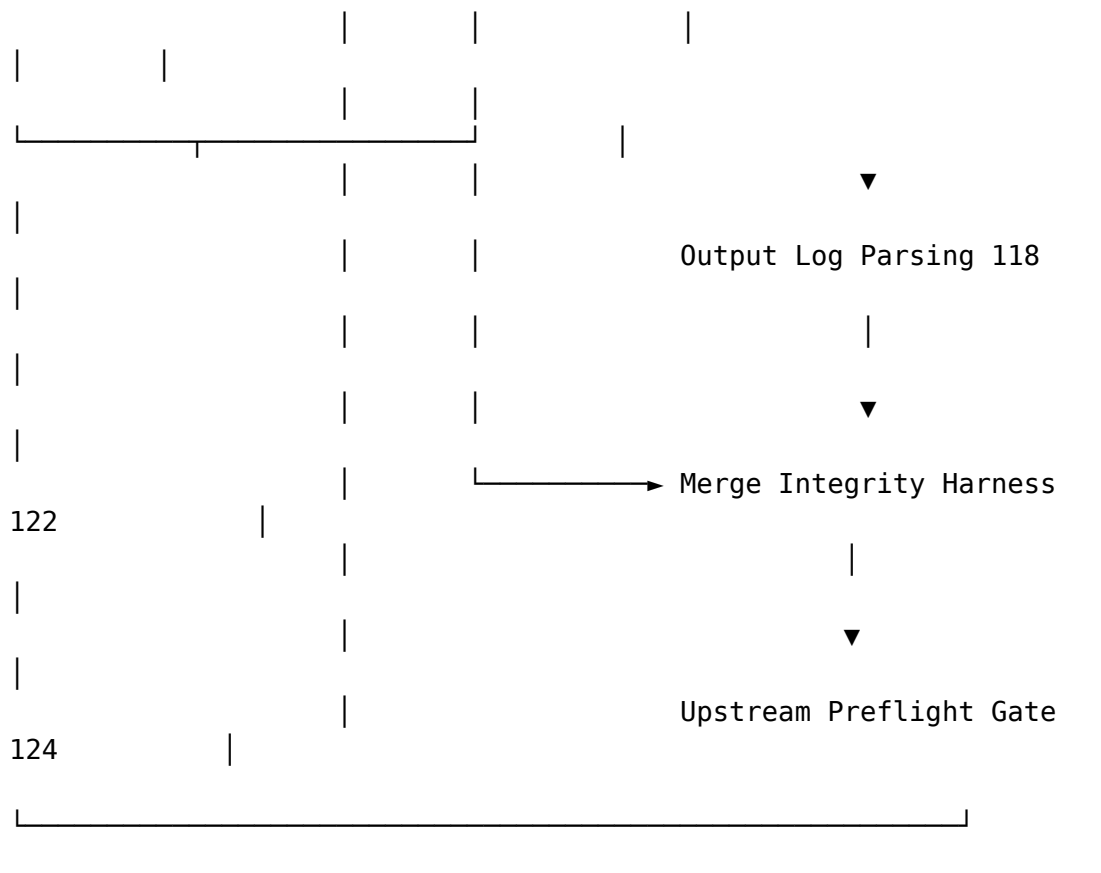
FIG. 11 is a federated operational contract diagram with vault instances, federated link syntax parser, lazy remote resolver, cross-system transaction publish, and integrity validator (numerals 1100–1110).

FIG. 12 is an observable read isolation (ORI) diagram with shared resource delivery log, observable-read tracker, stale-mutation gate, and commit rejection (numerals 1200–1210).

FIG. 13 is a runtime constraint synthesis diagram with static invariants, constraint evaluator, runtime constraint envelope, non-compliant path suspension, and preflight rejection list (numerals 1300–1310).

FIG. 1 — Deployment Loop (Block Diagram)





Detailed Description of the Invention

The following description references the accompanying drawings and the reduction-to-practice implementation. Like reference numerals designate like elements throughout.

Glossary of Defined Terms

The following terms are used throughout this specification and carry the meanings ascribed below:

- **Interaction graph $G = (V, E)$:** A directed graph wherein each vertex V represents a discrete code file or reactive intent node (HTTP route, CLI command, database mutation, inline guard condition) and each directed edge E represents an import dependency, invocation path, or wikilink reference between vertices.
- **Dark bridge:** A directed edge whose information-flow curvature metric $\kappa \leq -0.30$ (or an empirically tuned value within a range of -0.20 to -0.50), indicating an extreme information compression bottleneck that requires structural isolation between coupled modules.
- **Shadow bridge:** A directed edge with $\kappa < 0$ but above the dark bridge threshold, indicating latent coupling warranting monitoring but not mandatory atomic welding.

- **Atomic weld block:** A set of repository modules fused by union-find clustering into a single-work-unit execution block, wherein all files in the block are assigned to one agent and parallel assignment within the block is forbidden.
 - **Gateway mode (Contraction Rule):** An alternative partitioning mode wherein dark bridges are transformed into HANDOFF_ONLY transactional boundaries rather than atomic weld blocks, enabling parallel execution across formerly serialized modules with cross-boundary mutations routed through the council transaction bus.
 - **Agent Intent Vector (GIV):** An immutable permission envelope assigned to each agent workspace specifying allowed write paths, denied paths, sibling-denied paths, denied commands, and structural enforcement tokens.
 - **Council transaction bus:** An append-only ledger infrastructure (`.gaijinn/bridge/council.md` or `council.jsonl`) residing in a GIV-protected invariant path outside all worker write scopes, through which cross-boundary mutation requests and receipts are published as delimited handoff tickets.
 - **Honest convergence score:** A composite structural convergence metric computed from validation pass rate, handoff isolation success, transaction-bus synchronization status, and merge dispositions, wherein no-score padding is applied when validation passes but an agent produces zero fresh file deltas.
 - **Preflight gate:** An upstream API endpoint (`/api/v1/preflight`) that evaluates validated worker manifests and handoff queue state, returning a merge-cleared or merge-rejected status code as a hard prerequisite to repository branch integration.
-

System 100 — Overall Architecture

System 100 comprises four cooperating subsystems: (1) a geometric Ollivier-Ricci code curvature engine 106; (2) a GIV non-invasive enforcement and isolation substrate layer 112–116; (3) an asynchronous runtime council transaction bus 118; and (4) a merge pipeline governance and honest convergence analytics module 122–124.

System 1 — Geometric Ollivier-Ricci Code Curvature Engine (106)

The curvature engine 106 non-invasively maps a repository by constructing a directed graph $G = (V, E)$, where:

- **V** represents discrete code files and/or Layer 1 reactive intent nodes mapping HTTP paths, CLI commands, database mutations, and inline guard conditions; and
- **E** represents import dependencies, invocation paths, or wikilink references between modules.

Each vertex carries metadata including `capability_level` and `side_effect_score`, derived as follows:

- **Capability level** is a normalized integer on a scale of 1–10 computed during AST parsing by counting distinct operational entry points per module: HTTP route registrations, CLI command exports, public function declarations, and database migration invocations. Modules exposing 0–1 entry points receive level 1–2; modules with 2–5 entry points receive level 3–5; modules with 6–15 entry points receive level 6–8; modules exceeding 15 entry points receive level 9–10. Path-segment keywords in URL routes provide additional up-weighting: paths containing `api/`, `admin/`, or `billing/` are boosted by +1.
- **Side-effect score** is a normalized integer on a scale of 0–10 computed by scanning source files for database schema mutation operations (`CREATE TABLE`, `ALTER TABLE`, `INSERT`, `UPDATE`, `DELETE`), filesystem write operations (`os.write`, `open(..., 'w')`), network service registrations, and external API call invocations. Each detected mutation increments the score by 1; scores above 10 are capped. Modules with zero detectable side effects receive a base score of 0.25 to distinguish them from untagged modules.

Each directed edge ($u \rightarrow v$) undergoes the following transformation pipeline.

Step 1 — Distribution Generation (numeral 202)

For edge $u \rightarrow v$, the engine constructs discrete probability distributions over the immediate successors (neighborhoods) of u and v . Each neighborhood node n receives a risk weight:

$$\text{Weight}(n) = \alpha \cdot \text{Capability}(n) + \beta \cdot \text{SideEffect}(n)$$

where α and β are human-calibrated priority scalars. In the reference implementation, $\alpha = 0.70$ and $\beta = 0.30$. Weights are normalized to a probability mass function over the neighborhood support set. When a node has no outgoing successors, the distribution collapses to a self-loop at the node.

Step 2 — Cost Matrix Construction (numeral 204)

The engine builds a cost matrix spanning the support nodes of the two distributions by measuring undirected shortest-path lengths on an undirected projection of G . When two support nodes are disconnected, the engine injects a defensive fallback constraint distance equal to $|V|$ (total node count). This value represents the maximum possible graph distance—no two nodes in a graph of $|V|$ vertices can be separated by a shortest path exceeding $|V|$ in an unweighted graph—and mathematically enforces isolation boundaries by guaranteeing a Wasserstein cost that yields $\kappa \leq -0.30$, automatically classifying the edge as a structural bottleneck.

Step 3 — Wasserstein Distance Computation (numeral 206)

The engine invokes a discrete optimal transport algorithm—earth-mover distance via `ot.emd2`—to compute the Wasserstein-1 transport cost W between the neighborhood probability distributions given the cost matrix.

Step 4 — Curvature Valuation κ (numeral 208)

Geometric edge curvature is computed as:

$$\kappa = 1.0 - \frac{W}{\text{dist}_G(u, v)}$$

where `dist_G(u, v)` is the directed shortest-path length from `u` to `v` in `G`.

Edges where $\kappa \leq -0.30$ (or an empirically tuned value within a range of -0.20 to -0.50) exhibit extreme information compression bottlenecks and are classified as **dark bridges**. Edges where $\kappa < 0$ are classified as **shadow bridges**. A supplemental **risk-jump** classifier flags edges where a low-context source node (capability ≤ 2.0 , side-effect ≤ 0.25) transitions to a high-capability, high-side-effect target (capability ≥ 5.0 , side-effect ≥ 3.0), forcing κ to a minimum of -1.0 regardless of geometric curvature.

Step 5 — Structural Gravity (numeral 210)

Independently of edge curvature, each vertex receives a normalized structural gravity score combining in-degree, out-degree, capability level, and side-effect score. Vertices below a gravity hard floor (e.g., `0.20`) trigger automatic rejection from autonomous write assignment.

Step 6 — Gateway Mode Override (numeral 212)

Legacy mode: Dark-bridge-connected components are merged via union-find into atomic weld blocks. All files in a block receive a single work unit; parallel assignment within the block is forbidden (**Surgery Rule**).

Gateway mode: Dark bridges become `HANDOFF_ONLY` gateway edges rather than atomic welds. Work units retain non-overlapping `allowed_paths`; cross-boundary mutations traverse the transaction bus (System 3). An audit subsystem emits a parallel efficiency matrix quantifying `atomic_weld_blocks`, `largest_atomic_weld_file_count`, `files_unlocked_from_atomic_welds`, and `handoff_gateway_count`.

Measured result (171-node monorepo): Legacy: 2 weld blocks, 40-file largest cluster, 44 files serialized. Gateway: 0 weld blocks, 44 files unlocked, 92 handoff gateways, 54 work units, max parallel workers 47.

Subgraphs linked only by edges with $\kappa \geq 0$ may be allocated to separate concurrent execution lanes (**Contraction Rule**).

System 2 — GIV Non-Invasive Enforcement & Isolation Substrates (112–116)

Each worker `w` receives a frozen **Agent Intent Vector (GIV)** permission envelope 112 comprising:

Field	Semantics
<code>worker_id</code>	Unique worker identifier
<code>allowed_paths</code>	Posix-relative paths the worker may mutate (whitelist)
<code>denied_paths</code>	Explicit write prohibitions

Field	Semantics
sibling_denied_paths	Paths owned by sibling workers (derived from blueprint)
denied_commands	Shell commands forbidden in output log audit (e.g., <code>git push</code>)
structural_tokens	Boolean flags: <code>SCOPE_STRICT</code> , <code>NO_SIBLING_TRESPASS</code> , <code>HANDOFF_ONLY</code>
invariants	Protected governance artifact paths

Construction validates non-empty `allowed_paths` and canonical deterministic serialization.

Isolation Substrates (114, 116)

Workers execute in one of:

- **Git worktree isolation (114):** Disposable branch (e.g., `gaijinn/worker-N`) with path-scoped writes; or
- **Filesystem copy sandbox (116):** Directory copy without `.git` metadata; deltas detected via `filecmp` byte comparison and content hash against a session baseline directory.

Trespass Detection (numeral 414)

At merge validation, the harness computes `changed_files(worker_dir, base_ref, baseline_dir, scope_paths)` and invokes trespass detection: any changed path not matching `allowed_paths` and not denied-only constitutes a **trespass**. Trespass is merge-blocking, not advisory. Workers are classified as `dirty`, `blocked`, `completed`, or `already_merged`.

Protected invariant paths (e.g., project governance artifacts under `.gaijinn/`) cannot be modified by workers unless explicitly allowlisted.

System 3 — Asynchronous Runtime Council Transaction Bus (118)

When an isolated agent requires a mutation in a sibling-owned path, direct modification is prevented by the GIV sibling-denied blocklist. The agent instead interacts with an append-only transaction bus.

Ticket Emission (numeral 402)

The agent prints a delimited **HANDOFF_TRANSACTION_REQUEST** block to its output `.log` standard stream:

```
++++ GAIJINN_HANDOFF_TICKET_START ++++
{ JSON payload }
++++ GAIJINN_HANDOFF_TICKET_END ++++
```

Payload schema:

- `ticket_id` — deterministic identifier (prefix TX-HT-)
- `source_worker_id`
- `target_work_unit_id`
- `target_file`
- `required_mutation_context`
- `status`, `timestamp`, optional `resolution_commit`

Scaffold Hardening (numeral 404)

At collection phase, the pipeline parses worker logs and screens payloads for scaffold template tokens (e.g., `<PATH_TO_FILE>`, `<TARGET_WORK_UNIT_ID>`, angle-bracket placeholders). Scaffold echoes are purged as prompt noise.

Receipt and Synchronization (numeral 406)

The target worker executes the mutation within its allowed paths and appends a **HANDOFF_TRANSACTION_RECEIPT** to the council ledger. The ledger is stored under `.gaijinn/bridge/council.md` (or `council.jsonl`), a governance artifact path designated as a **protected invariant** in every Agent Intent Vector permission envelope. Workers are explicitly forbidden from modifying the ledger via `denied_paths` and `invariants` fields; the ledger exists permanently outside all worker write scopes. This architecture ensures the transaction bus is tamper-proof against failing or compromised agent workspaces. Tickets persist in `handoff-queue.json`, also under `.gaijinn/`.

The merge engine enforces:

$$\text{Sync_Status} = (\text{Tickets_Raised} = \text{Tickets_Resolved}) \wedge (\text{Pending_Queue} = \emptyset)$$

Exemplar: TX-HT-6D0B24 — billing scope worker requested API preflight extension; resolved by sibling worker; bus synchronized; zero source-worker sibling writes.

System 4 — Merge Pipeline Governance and Honest Convergence Analytics (122–124)

The merge engine evaluates completed agent worktrees through a four-phase pipeline:

1. **collect** — Per-worker status, changed files, diff summary, handoff queue sync.
2. **validate-worker** — Gates: path allowlist, scope isolation, denied commands, handoff protocol, scoped acceptance tests, invariants, bus integrity.
3. **merge-grid** — Topological merge order respecting blueprint `depends_on`; copy-mode file propagation or git branch merge.
4. **governance synthesis** — Structural convergence score with invariant breakdown in `governance.json`.

Honest Convergence (numeral 410)

The system does **not** pad convergence when validation passes but merge detects zero fresh deltas. Copy-mode workers with no filesystem changes and completion-ledger matches receive `already_merged` disposition rather than `merge-blocked` status, yielding honest fractional convergence (e.g., **0.8889**) rather than vanity **1.0**.

Convergence aggregates: `validation_pass_rate`, `handoff_isolation`, `transaction_bus_synchronized`, `merge.no_blocked`, `merge.no_conflicted`, and worker merge counts.

Preflight Gate (numeral 124)

POST `/api/v1/preflight` evaluates `validated.json` and `handoff-queue.json`:

- `PREFLIGHT_CLEARED` (HTTP 200) — safe to integrate
 - `PREFLIGHT_REJECTED` (HTTP 422) — trespass or unresolved transaction tickets
-

Claims

The following claims are draft language for attorney conversion. Brand-neutral terminology is used throughout.

Claim numbering note: Claims 2–7 are dependent on Claim 1. Claim numbers 13–15, 17–19, 21–24, 26–29, 31–33, 35–37, 39–41, and 43–45 are reserved for dependent claims of their respective independent families. Full dependent chains appear in the companion filing addendum (`PROVISIONAL-PATENT-TECHNICAL-SPECIFICATION.md` PART XI).

Independent Claim 1 — Core Geometric Orchestration Method

1. A computer-implemented method for parallel software engineering orchestration and risk mitigation, comprising:

- a. transforming, by an intake scanner module, a target code repository into an interaction graph mapping repository modules as a set of vertices and module dependencies as a set of directed edges;
- b. generating, for a directed edge $u \rightarrow v$ connecting a source node u and a target node v within the interaction graph, a pair of neighborhood probability distributions spanning the neighborhood successors of u and v respectively, wherein each neighborhood node is weighted by a composite risk score combining an application capability level metric and an operational side-effect score;
- c. constructing a cost matrix across the neighborhood distributions by calculating undirected shortest-path lengths between neighborhood support nodes on an undirected projection of the interaction graph, and injecting a total-node-count fallback distance constraint when a pair of support nodes is disconnected;

- d. computing a Wasserstein optimal transport distance between the pair of neighborhood probability distributions based on the cost matrix via an earth-mover distance calculation;
 - e. evaluating an information-flow curvature metric κ for the directed edge, wherein the curvature metric is calculated as $\kappa = 1 - (W / \text{dist}_G(u, v))$, where W is the computed Wasserstein optimal transport distance and $\text{dist}_G(u, v)$ is a base graph distance separating node u and node v ;
 - f. identifying a subset of directed edges whose curvature metric κ falls below a predetermined negative risk threshold as structural bottleneck constraints designated as dark bridges, wherein edges with $\kappa < 0$ but above said threshold are classified as shadow bridges;
 - g. partitioning the target code repository into a plurality of isolated work units by executing a union-find clustering pass that fuses repository modules joined by the identified dark bridges into an atomic single-worker execution block while partitioning uncoupled subgraphs into separate concurrent execution lanes, wherein each edge connecting nodes in separate lanes exhibits a curvature metric $\kappa \geq 0$;
 - h. spawning a plurality of parallel autonomous agent workspaces respectively mapped to the plurality of isolated work units, wherein each workspace is injected with an absolute write-scope contract defining an allowed path whitelist and a denied sibling-worker path blocklist; and
 - i. validating, by a merge integrity harness, a file delta generated by an individual autonomous agent workspace against its respective write-scope contract to completely block integration of the file delta into a primary repository branch upon detection of an un-allowlisted path trespass.
-

Dependent Claims 2–6 (Narrowing Independent Claim 1)

- 2. The method of claim 1, wherein the information-flow curvature metric is an Ollivier-Ricci curvature value, and wherein the predetermined negative risk threshold is an empirical hard floor of $\kappa = -0.30$, or an empirically tuned value within a range of $\kappa = -0.20$ to $\kappa = -0.50$.
- 3. The method of claim 1, further comprising an alternative gateway processing mode wherein the identified shadow bridges are transformed into non-overlapping transactional boundaries designated as HANDOFF_ONLY gateway edges in lieu of executing the union-find clustering pass, and generating an automated parallel efficiency matrix tracking files unlocked from atomic serialization.
- 4. The method of claim 1, wherein the absolute write-scope contract further comprises a denied-command blocklist, and further comprising scanning a worker execution output log stream to intercept and block integration upon detecting a prohibited repository push operation.
- 5. The method of claim 1, wherein when an autonomous agent workspace requires an architectural modification within a file belonging to the denied sibling-worker path blocklist, the workspace writes a delimited transaction request token containing a

unique transaction identifier, a target file path, and a required mutation context to an append-only council ledger file.

6. The method of claim 5, further comprising parsing the append-only council ledger file at a collection phase, filtering out scaffold template tokens from the ledger to eliminate prompt placeholder artifacts, tracking a transaction receipt appended to the ledger by a target worker, and blocking repository integration unless a transaction bus synchronization invariant is satisfied, wherein the invariant requires all raised transaction identifiers to match resolved transaction receipts.

7. The method of claim 1, further comprising: detecting a risk-jump edge transition wherein a source node exhibits a capability level at or below a first threshold and an operational side-effect score at or below a second threshold and a target node exhibits a capability level at or above a third threshold and an operational side-effect score at or above a fourth threshold; and forcing the curvature metric κ of said risk-jump edge to a minimum floor value regardless of the computed Wasserstein optimal transport distance.

Independent Claim 8 — Operational Contract Compilation

8. A computer-implemented method for parallel software engineering orchestration and risk mitigation, comprising:

- discovering, by a non-invasive capability inventory module, a plurality of operational endpoints and execution dependencies exposed by a target software codebase without modifying host-system source code;
 - constructing, by a topology compiler, a multi-layer interaction graph from the discovered endpoints and dependencies, the graph mapping physical endpoints to a set of reactive intent nodes and inferring emergent lifecycle chains between said intent nodes;
 - compiling, by a blueprint compiler, a machine-readable Certified Operational Contract (COC) from said multi-layer interaction graph, the contract comprising a plurality of partitioned work units, explicit non-overlapping write-scope assignments, and per-agent permission vectors; and
 - enforcing, by a merge integrity harness, a file delta generated by an autonomous agent workspace against its respective per-agent permission vector at a codebase integration phase to completely block non-compliant code modifications prior to repository branch integration.
-

Independent Claim 9 — Gateway Mode Parallel Execution (Contraction Rule)

9. A computer-implemented method for unlocking parallel execution lanes in a geometrically partitioned software repository undergoing concurrent autonomous agent modification, comprising:

- a. identifying, by a geometric curvature engine, a set of structural bottleneck edges in a directed repository interaction graph whose information-flow curvature metric κ falls below a predetermined negative risk threshold, said bottleneck edges representing latent coupling between otherwise independent repository modules;
- b. transforming, by a blueprint planner executing in gateway processing mode, the identified structural bottleneck edges into non-overlapping transactional gateway boundaries designated as HANDOFF_ONLY gateway edges in lieu of atomic union-find weld blocks, each gateway boundary defining a cross-worker mutation path that is resolvable only through an asynchronous transaction bus rather than direct filesystem write;
- c. partitioning the target code repository into a plurality of isolated work units whose allowed write paths are pairwise non-overlapping across the HANDOFF_ONLY gateway boundaries, wherein subgraphs linked exclusively by edges with $\kappa \geq 0$ are allocated to separate concurrent execution lanes;
- d. spawning a plurality of parallel autonomous agent workspaces respectively mapped to said partitioned work units, wherein each workspace receives an immutable Agent Intent Vector permission envelope forbidding direct modification of gateway-bounded sibling paths; and
- e. emitting, by an automated audit subsystem, a parallel efficiency matrix quantifying files unlocked from atomic serialization as a difference between a legacy union-find atomic weld file count and a gateway-mode serialized file count.

10. The method of claim 9, wherein the parallel efficiency matrix comprises metrics selected from: atomic weld block count, largest atomic weld file count, files unlocked from atomic welds, handoff gateway count, maximum concurrent parallel workers, and a serialization ratio.

11. The method of claim 9, wherein the gateway processing mode is activated by a configuration flag distinct from a default legacy union-find mode, and further comprising selecting between legacy and gateway modes based on repository size or detected bottleneck topology.

Independent Claim 12 — Non-Invasive Capability Extraction

12. A non-invasive capability extraction system for inventorying application boundaries of a software repository, comprising:

- one or more processors configured to execute an extraction scanner module;
- said scanner module configured to select an extraction modality from the group consisting of: abstract-syntax-tree parsing of source files, OpenAPI specification ingestion, Swagger specification ingestion, and Model Context Protocol (MCP) tool manifest parsing;
- said scanner module configured to traverse a target repository filesystem while honoring a local exclude pattern layout, emitting a common capability model file containing an inventory of file nodes annotated with a capability level metric and

an operational side-effect score;

- an intent-node extractor that maps network routes, command-line interface instructions, database mutations, and inline conditional guards to a reactive graph ledger; and
 - a serialized graph artifact writer that persists the capability records and directed reference edges to a machine-readable interaction graph file without mutating host-system source code.
-

Independent Claim 16 — Capability Topology Governance

16. A computer-implemented method for governing concurrent system execution as a strongly typed capability topology, comprising:

- representing, in a directed capability graph, each operational component as a typed node annotated with a unique capability identifier, a functional resource cluster designation, and a normalized risk score;
 - representing each valid execution path as a directed edge between a source capability node and a target capability node, wherein the edge is annotated with a transition classification derived from reachability analysis;
 - validating, by a blueprint validator module, that a plurality of concurrent execution scopes mapped to separate agent workspaces are pairwise non-overlapping; and
 - rejecting, by the blueprint validator module, compilation of a repository build manifest upon detecting an ambiguous or overlapping filesystem write assignment between any pair of concurrent agent workspaces.
-

Independent Claim 20 — Structural Gravity Certification

20. A computer-implemented method for constraining autonomous execution paths using architectural geometry, comprising:

- computing, for each node in a software repository dependency graph, a normalized structural gravity score as a weighted combination of node in-degree, node out-degree, capability level, and side-effect reach;
 - identifying a subset of nodes whose computed structural gravity score falls below an empirical gravity hard floor;
 - flagging said subset of nodes within a governance metadata record as automatically rejected from autonomous write assignment; and
 - compiling an execution blueprint that isolates the automatically rejected nodes behind dedicated human calibration review layers, whereby autonomous modifications to the flagged nodes are blocked at a preflight execution phase.
-

Independent Claim 25 — Runtime Enforcement of Certified Agent Boundaries

25. A runtime security system for enforcing certified agent boundaries during parallel autonomous software engineering, comprising:

- a supervisory daemon configured to poll a repository metrics manifest and execute validation routines upon each write to the manifest;
 - an authorization gate matcher configured to load an immutable Agent Intent Vector (GIV) permission envelope for an active agent workspace, the envelope defining allowed paths, denied paths, sibling-worker paths, prohibited commands, and structural enforcement tokens;
 - a drift detection module configured to capture file deltas generated within the agent workspace by computing a diff relative to a baseline reference;
 - a trespass classifier configured to parse the captured file deltas and block integration of the workspace upon detecting a file modification outside the allowed paths whitelist; and
 - an automated circuit breaker module configured to transition a global grid sprint readiness state to a blocked execution mode upon detecting a tripped safety marker selected from the group consisting of: an active automatic rejection flag, a non-zero shadow bridge count, or an incomplete worker manifest.
-

Independent Claim 30 — Self-Certifying Software Architecture

30. A computer-implemented method for continuous self-certification of a software system undergoing autonomous agent modifications, comprising:

- scanning, by an intake module after an execution sprint, a target repository to generate an interaction graph and a curvature metrics manifest reflecting an active architectural surface;
 - comparing agent-generated filesystem deltas against an append-only completion ledger recording prior content hashes of successfully integrated work units;
 - classifying a worker workspace that exhibits zero fresh filesystem deltas and whose assigned work unit matches an existing completion ledger entry with an already-integrated merge disposition;
 - computing a composite structural convergence score across a plurality of parallel workspaces as a function of validation pass rates, handoff isolation success, transaction-bus synchronization, and merge dispositions, wherein the score prevents metric padding by registering the already-integrated merge disposition as an honest fractional value; and
 - triggering an automated re-certification event when the computed structural convergence score falls below a configured certification threshold.
-

Independent Claim 34 — Federated Operational Contract Governance

34. A computer-implemented method for federated governance of operational contracts across multi-system boundaries, comprising:

- deploying a plurality of independent software repository instances under a shared vault root, each instance maintaining a localized Certified Operational Contract and an active deploy manifest;
 - parsing cross-system references encoded in a federated link syntax, the syntax specifying a remote vault identifier and a targeted remote capability record;
 - resolving cross-system links lazily at query runtime by fetching data from the remote deploy manifests, thereby preventing redeployment of a local system when a remote vault configuration changes;
 - publishing a cross-boundary mutation requirement from a source system's agent workspace to a shared append-only council transaction bus ledger accessible by a target system's workspace; and
 - validating cross-system transaction integrity prior to integration by asserting that all transaction request tickets published to the bus ledger match an explicitly recorded target transaction receipt.
-

Independent Claim 38 — Observable Read Isolation (ORI)

38. A concurrency coordination system for multi-agent software engineering environments, comprising:

- a delivery log data structure recording a monotonically increasing resource version and an authorized read timestamp for a plurality of shared repository resources;
 - an observable-read tracker module configured to intercept an agent read operation against a shared repository resource and append an entry to the delivery log binding an agent identifier, a resource path, an observed version, and a UTC read timestamp;
 - a stale-mutation gate configured to compare an agent's last recorded observed version against a current active resource version in the delivery log prior to committing a workspace mutation; and
 - a rejection module configured to block a commit write operation from the workspace when the agent's last recorded observed version is strictly less than the current active resource version in the delivery log.
-

Independent Claim 42 — Runtime Constraint Synthesis

42. A computer-implemented method for dynamically synthesizing contract constraints to govern autonomous workflow execution paths, comprising:

- loading a Certified Operational Contract comprising a set of static structural invariants, domain rules, and per-agent permission vectors;
 - ingesting a real-time context stream comprising a plurality of runtime signals selected from the group consisting of: metrics-manifest safety state, pending handoff ticket counts, worker output-log error patterns, and session-scoped project root overrides;
 - synthesizing, by a constraint evaluator module, a runtime constraint envelope by mapping the real-time context stream against the static structural invariants to identify cross-cutting boundary conflicts;
 - applying the synthesized runtime constraint envelope at a grid execution phase to suspend non-compliant multi-agent workflow paths; and
 - emitting a deterministic rejection-reason list and a merge-rejected HTTP status code via a preflight API gate when any synthesized constraint is breached.
-

Cross-Cutting Claims

46. A non-transitory computer-readable medium storing instructions that, when executed by one or more processors, cause the processors to perform the method of any of claims 1, 8, 16, 20, 30, 34, 38, or 42.

47. A system comprising one or more processors and a memory storing instructions that, when executed, implement operations of any of claims 12, 25, or 38.

Abstract of the Disclosure

An execution system and computer-implemented method are disclosed for governed parallel software agent orchestration using geometric repository analysis. An intake module maps a codebase into an interaction graph where nodes represent application components and edges define dependencies. A geometric analyzer evaluates structural gravity and Ollivier-Ricci curvature across edge distributions using Wasserstein optimal transport distances to isolate hidden structural bottlenecks. A blueprint compiler partitions the graph via union-find tracking to weld highly coupled modules into atomic single-worker units while splitting uncoupled subgraphs into parallel execution lanes. Concurrently spawned agent workspaces are restricted by an immutable Agent Intent Vector defining strict path allowlists. Sibling-path updates are routed through an asynchronous transaction bus via delimited log tokens. A merge integrity harness evaluates agent deltas against the contract definitions, verifying transaction bus synchronization and blocking integration upon detecting a sibling path trespass.

(148 words)

Reduction to Practice — Empirical Evidence

Metric	Phase 1 (12-node service)	Phase 2 (171-node monorepo)
Code nodes	12	171
Shadow bridges / gateways	—	92
Concurrent workers	2	4
Atomic weld blocks (gateway audit)	0	0
Largest legacy weld (files)	40	40
Files unlocked from welds	44	44
Handoff ticket	TX-HT-84348F	TX-HT-6D0B24
Validation pass rate	1.0	1.0
Trespass violations	0	0
Transaction bus synchronized	true	true
Convergence (honest)	1.0	0.8889

Artifacts: `governance.json`, `handoff-queue.json`, `validated.json`, `audit-report.json`.

Filing Checklist (Assignee Action Items)

- ☐ Engage registered patent attorney (software / AI systems experience)
- ☐ Complete inventor assignment and entity status (Neural Draft LLC)
- ☐ Prepare FIG. 1–4 formal drawings with reference numerals
- ☐ Attach this specification + selected code excerpts as PDF bundle
- ☐ File provisional application via USPTO Patent Center (patentcenter.uspto.gov)
- ☐ Record filing date; mark collateral **Patent Pending**
- ☐ Calendar 12-month non-provisional conversion deadline

Neural Draft LLC. Trade secret notice applies. Do not distribute externally without NDA and attorney approval.